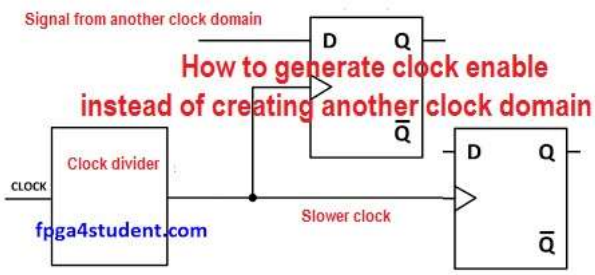
# How to generate a clock enable signal on FPGA

**This post is about to tell you how to generate a clock enable signal (*not gated clocks*) to drive another logic using the same clock domain instead of creating another clock (using clock dividers or clock gating) possibly causing FPGA timing issues (as created by non-dedicated FPGA clock generators) or clock domain crossing problems such as metastability, data loss, and data incoherency if it is not taken care.**




It is recommended by both Xilinx and Altera to use clock enable, which can help save FPGA clock resources and improve FPGA timing characteristics and timing analysis of the design.
The multi-clock domain crossing problem is very common in digital logic design. When you interface signals between different clock domains, metastability and data loss are likely to happen due to setup/hold time violations. Check HERE to understand what metastability is and why it occurs. The most popular way to prevent this is by using multi-Flip-Flop synchronizers to synchronize the input signals from another clock domain.
To avoid the FPGA timing issues or clock domain crossing issues, it is recommended to generate a slow clock enable signal instead of creating another slower clock (using clock dividers or clock gating) to drive another logic part of your design.
For example, in your FPGA, there is a 50MHz clock available, but you want to drive another part of your design using a slower clock of 1KHz. Instead of creating another clock of 1KHz, you should create a 1KHz clock enable signal.
**Below is an example VHDL code for generating the slow clock enable signal:**

```
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects,
-- Generate clock enable signal instead of creating another clock domain
-- Assume that the input clock : clk_50MHz
signal clock_1KHz_enable  : std_logic;
signal counter : std_logic_vector(15 downto 0):=x"0000";
constant DIVISOR: std_logic_vector(15 downto 0):= x"C34F";
-- Generate the slow enable signal instead of creating another clock of 1KHz
--
process(clk_50MHz)
begin
  if(rising_edge(clk_50MHz)) then
    if(counter = DIVISOR) then
      counter <= x"0000";
      clock_1KHz_enable <= '1';
    else
      clock_1KHz_enable <= '0';
      counter <= counter + x"0001";
    end if;
  end if;
end process;
-- Use the same clock and the slow clock enable signal above
-- to drive another part of the design to avoid domain crossing issues
process(clk_50MHz)
begin
  if(rising_edge(clk_50MHz)) then
    if(clock_1KHz_enable = '1') then
```

```
      -- Add your logic here
      -- It will be executed like a process of 1Khz clock
    end if;
  end if;
end process;
```

By creating the clock enable signal instead, all the logic in your design are driven by the same clock so that you won't need to worry about the FPGA timing issues or the multi-clock domain crossing problems. You can change the DIVISOR constant value to get any clock enable frequency that you want. Another Verilog example code for generating a slow clock enable signal instead of creating another clock domain: here.

Another VHDL example code here and Verilog example code here.

**You may like this:**

1. **What is an FPGA? How VHDL works on FPGA**
2. **VHDL code for FIFO memory**
3. **VHDL code for FIR Filter**
4. **VHDL code for 8-bit Microcontroller**
5. **VHDL code for Matrix Multiplication**
6. **VHDL code for Switch Tail Ring Counter**
7. **VHDL code for digital alarm clock on FPGA**
8. **VHDL code for 8-bit Comparator**
9. **How to load a text file into FPGA using VHDL**
10. **VHDL code for D Flip Flop**
11. **VHDL code for Full Adder**
12. **PWM Generator in VHDL with Variable Duty Cycle**
13. **VHDL code for ALU**
14. **VHDL code for counters with testbench**
15. **VHDL code for 16-bit ALU**
16. **Shifter Design in VHDL**
17. **Non-linear Lookup Table Implementation in VHDL**
18. **Cryptographic Coprocessor Design in VHDL**
19. **Verilog vs VHDL: Explain by Examples**
20. **VHDL code for debouncing buttons on FPGA**
21. **VHDL code for Traffic light controller**
22. **VHDL code for debouncing buttons on FPGA**
23. **VHDL code for Traffic light controller**
24. **VHDL code for a simple 2-bit comparator**
25. **VHDL code for a single-port RAM**
26. **VHDL code for Car Parking System using FSM**
27. **VHDL coding vs Software Programming**

**Recommended Verilog projects:**

1. **What is an FPGA? How Verilog works on FPGA**
2. **Verilog code for FIFO memory**
3. **Verilog code for 16-bit single-cycle MIPS processor**
4. **Programmable Digital Delay Timer in Verilog HDL**
5. **Verilog code for basic logic components in digital circuits**
6. **Verilog code for 32-bit Unsigned Divider**
7. **Verilog code for Fixed-Point Matrix Multiplication**
8. **Plate License Recognition in Verilog HDL**
9. **Verilog code for Carry-Look-Ahead Multiplier**
10. **Verilog code for a Microcontroller**
11. **Verilog code for 4x4 Multiplier**
12. **Verilog code for Car Parking System**
13. **Image processing on FPGA using Verilog HDL**
14. **How to load a text file into FPGA using Verilog HDL**
15. **Verilog code for Traffic Light Controller**
16. **Verilog code for Alarm Clock on FPGA**
17. **Verilog code for comparator design**
18. **Verilog code for D Flip Flop**

19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.